

Education

[My University]

Bachelor of Science in Computer Science

Expected Dec 2028

Year 2

Technical Skills

Languages: Java, JavaScript, SQL

Frameworks: Spring Boot, Spring Security, Spring Data JPA, Spring Data JDBC

Databases & Messaging: PostgreSQL, Redis, RabbitMQ

Concurrency & Performance: Virtual Threads, JMM, MVCC, JMH, JCSstress, JFR

Tools & Platforms: Docker, Git, Azure, Maven, Gradle, gRPC, OAuth2, JWT, JUnit, Mockito

Projects

Clique | 5,400+ Maven downloads | **127 stars** | *Java* | [GitHub](#) · [Maven Central](#)

Active

- Published a dependency-free Java terminal styling library; 5,400+ Maven downloads, 127 GitHub stars.
- Supports GraalVM native image compilation, Unicode v16.0 emoji, and no-color.org compliance out of the box
- Shipped with RGB/gradient rendering, a full layout component system, and multiple built-in themes.

tx-map | *Java, JMH* | [GitHub](#)

- Implemented and benchmarked 6 transactional map variants from scratch (Optimistic, Pessimistic, Read Uncommitted, Copy-on-Write, Flat Combined, MVCC); found single combiner outperforms segmented - better batching amortization beats parallelism.
- Optimized MVCC map throughput from 2K to 4M+ ops/s through iterative JMH profiling; identifying GC pressure, write contention and boxed primitive allocations as primary bottlenecks.

Streamline | *Java 21, Spring Boot* | [GitHub](#) · [JitPack](#)

- Built a thread-safe SSE management library on Java 21 virtual threads; enables non-blocking concurrent event streaming without dedicated threads per connection.
- Designed registry pattern with bounded event history, configurable eviction policies (FIFO/LIFO/STRICT), automatic cleanup, and event replay with flexible predicate filtering for reconnecting clients.

EventDrop | *Spring Boot, Azure, RabbitMQ, Redis* | [GitHub](#) · [Live](#)

- Built a full-stack ephemeral file sharing PWA with a 5-layer cleanup system (Redis TTL, event-driven cascades, scheduled jobs, storage lifecycle policies) to prevent data leaks; deployed to Azure with quota enforcement.
- Solved a race condition in concurrent batch uploads using atomic validation under real load.

Research & Experimentation

vic-utils - Experimental Concurrency Primitives | *Java, JMH* | [GitHub](#)

- Benchmarked CSP channel implementations (lock-based, lock-free) showing a 50× throughput gap between variants (5M vs. 100K ops/s). Quantified how lock strategy dominates under contention.
- Designed a novel Optimistic Entity pattern combining version-based optimistic locking, single-threaded actor processing and event sourcing; built an actor model runtime similar to Akka with supervision trees using virtual threads.

Technical Writing

Dev.to | Concurrency & Performance | [dev.to](#)

- **“Improving A MVCC Transactional Map”** — Step-by-step profiling journey from ~2K to 4M+ ops/s: diagnosing GC pressure, epoch scan overhead and primitive boxing allocations
- **“A Practical Exploration of Transactional Map Implementations in Java”** — Deep dive into MVCC design: version chains, epoch reclamation variants, and tradeoffs between different MVCC version chain archetypes
- **“What Happens When You Give a Map Transactional Semantics”** — Overview of 5 isolation strategies with benchmark comparisons across read/write workloads.
- **“Memory Barriers and Happens-Before Guarantees”** — Deep dive into the Java Memory Model, hardware-level memory barriers, and happens-before rules.